

Inheritance

Inheritance :- It is one of the most important features of Object-Oriented Programming. The Capability of a class to derive properties and characteristics from another class is called inheritance.

Inheritance is a feature or a process in which, new classes are created from the existing classes. The new class created is called "Derived Class" or "Child Class" or "Sub Class" and the existing class is known as the "Base Class" or "Parent Class" or "Super Class".

When we say derived class inherits the base class, it means the derived class inherits all the properties of base class without changing the properties of base class and may add new features to its own derived class.

Why and when to use Inheritance.

- Inheritance is a term for reusing code by a mechanism of passing down information and behavior from a parent class to a child or subclass.
- Consider a group of vehicles. You need to create classes for Bus, Car, and Truck. The methods fuelAmount(), capacity(), applyBrakes(), will be the same for all three classes. If we create these classes avoiding inheritance then we have to write all of these functions in each of the three classes as shown below figure:-

Class Bus

fuelAmount()
capacity()
applyBrakes()

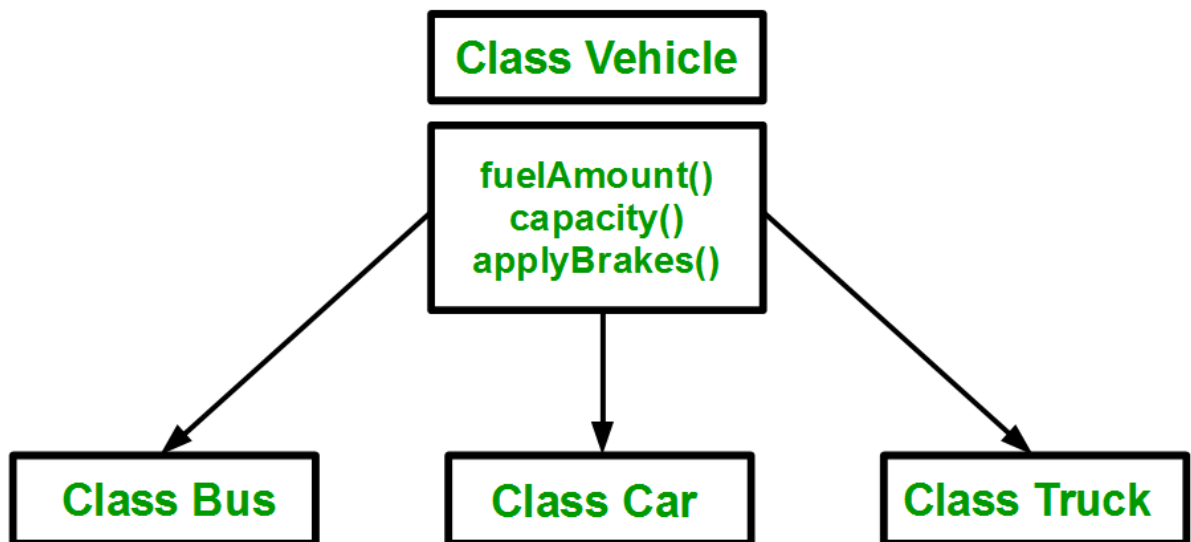
Class Car

fuelAmount()
capacity()
applyBrakes()

Class Truck

fuelAmount()
capacity()
applyBrakes()

- We can clearly see that the above process results in duplication of the same code 3 times. This increases the chances of error and data redundancy. To avoid this type of situation, inheritance is used.



- If you want the object to use all the behavior of the base class unless explicitly overridden, then inheritance is the simplest, least verbose, most straightforward way to express it.

- The main reason for using inheritance is not as a form of composition - it is so you can get polymorphic behavior. If you don't need polymorphism, you probably should not be using inheritance, at least in C++.

Available vs Accessible

Available : If a class Derived is inheriting from Base class then everything from Base class will be available/present in Derived class.

Accessible: everything is available in Derived class but private members are not accessible.

Example: bank balance of Father is available for Son by not accessible. Son has to ask Father for money. Only Father can access his account.

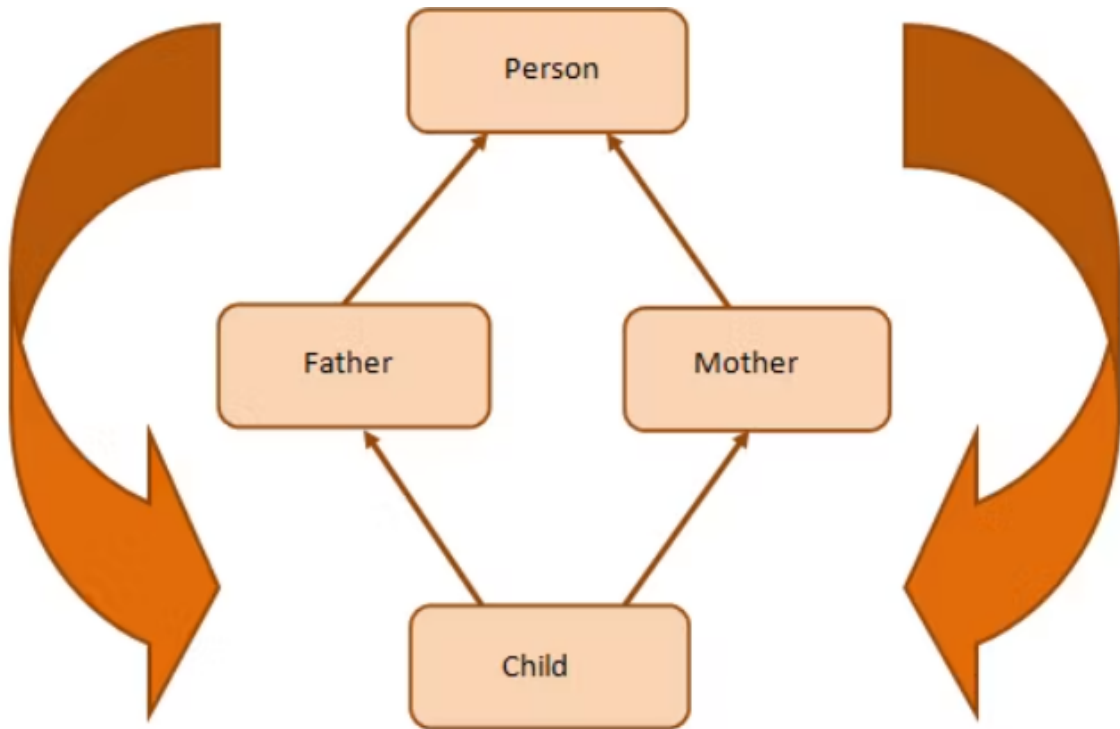
```
1 #include<bits/stdc++.h>
2 using namespace std;
3 // Example of inheritance
4 class Rectangle{
5 private:
6     int length;
7     int breadth;
8 public:
9     Rectangle(){length = 1,breadth=1;};
10    Rectangle(int l, int b){length = l,breadth=b;};
11    int getLength(){return length;};
12    int getBreadth(){return breadth;};
13    void setLength(int l){length = l;};
14    void setBreadth(int b){breadth = b;};
15    int area(){return length * breadth;};
16    int perimeter(){return 2 * (length + breadth);};
17    bool isSquare(){return length == breadth;};
18    ~Rectangle(){cout<<"Rectangle Destroyed";};
19 };
20 class Cuboid : public Rectangle{
21 private:
22     int height;
23 public:
24     Cuboid(int h){
25         height = h;
26     }
27     int getHeight() { return height; }
28     void setHeight(int h) { height = h; }
29     int volume() { return getLength() * getBreadth() * height; }
30 };
31 int main(){
32     Cuboid c(5);
33     c.setLength(10);
34     c.setBreadth(7);
35     cout << "Volume is " << c.volume() << endl;
36 }
37 //Output
38 Volume is 350
39 Rectangle Destroyed
40
```

Constructors in inheritance :-

- Constructors of base class are executed first then the constructor of derived class is executed.
- By default, a non-parameterised constructor of base class is executed.
- A parameterised constructor of base class must be called from derived class constructor.

```
1 // Example of calling parameterised constructor
2 #include <iostream>
3 using namespace std;
4 class Base{
5 public:
6     Base(){cout<<"Non-param Base"<<endl;}
7     Base(int x){cout<<"Param of Base "<<x<<endl;}
8 };
9 class Derived:public Base{
10 public:
11     Derived(){cout<<"Non-Param Derived"<<endl;}
12     Derived(int y){cout<<"Param of Derived "<<y<<endl;}
13     Derived(int x,int y):Base(x){
14         cout<<"Param of Derived "<<y<<endl;
15     }
16 };
17 int main(){
18     Derived d(5,10);
19     return 0;
20 }
21 //Output
22 Param of Base 5
23 Param of Derived 10
```

The Diamond Problem (Multiple-path inheritance) :- The Diamond problem occurs when a child class inherits from two parent classes who both share a common grandparent class.



As shown in the figure, class *Child* inherits the traits of class *Person* twice—once from *Father* and again from *Mother*. This gives rise to ambiguity since the compiler fails to understand which way to go.

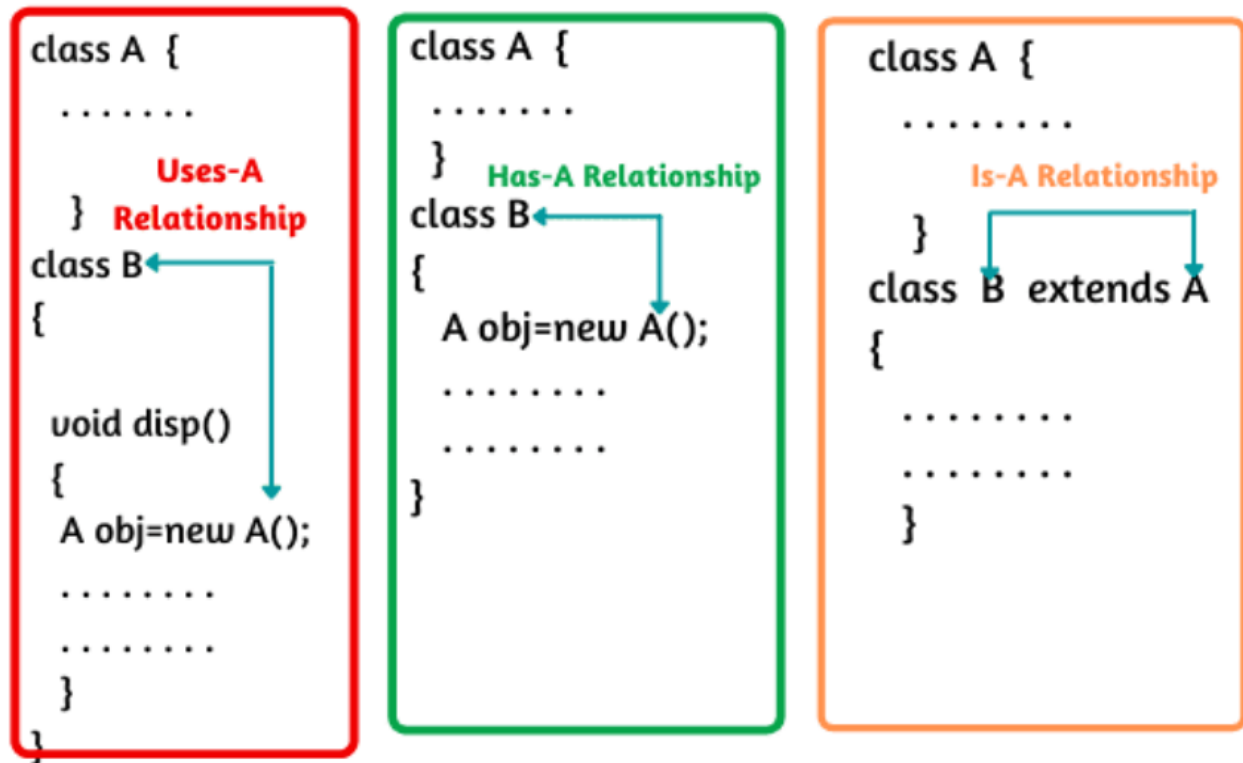


```
1 // Example of diamond problem
2 #include <iostream>
3 using namespace std;
4 class Person { //class Person
5 public:
6     Person(int x){cout << "Person::Person(int) called" << endl;}
7 };
8 class Father : public Person { //class Father inherits Person
9 public:
10     Father(int x):Person(x){
11         cout << "Father::Father(int) called" << endl;
12     }
13 };
14 class Mother : public Person{ //class Mother inherits Person
15 public:
16     Mother(int x):Person(x){
17         cout << "Mother::Mother(int) called" << endl;
18     }
19 };
20 class Child : public Father, public Mother{ //Child inherits Father and Mother
21 public:
22     Child(int x):Mother(x), Father(x) {
23         cout << "Child::Child(int) called" << endl;
24     }
25 };
26 int main(){
27     Child child(30);
28     return 0;
29 }
30
31 //Output
32 Person::Person(int) called
33 Father::Father(int) called
34 Person::Person(int) called
35 Mother::Mother(int) called
36 Child::Child(int) called
```

The solution to the diamond problem is to use the virtual keyword. We make the two parent classes (who inherit from the same grandparent class) into virtual classes in order to avoid two copies of the grandparent class in the child class

```
1 // Example of diamond problem
2 #include <iostream>
3 using namespace std;
4 class Person { //class Person
5 public:
6     Person(){cout << "Person::Person() called" << endl;}
7     Person(int x){cout << "Person::Person(int) called" << endl;}
8 };
9 class Father : virtual public Person { //class Father inherits Person
10 public:
11     Father(int x):Person(x){
12         cout << "Father::Father(int) called" << endl;
13     }
14 };
15 class Mother : virtual public Person{ //class Mother inherits Person
16 public:
17     Mother(int x):Person(x){
18         cout << "Mother::Mother(int) called" << endl;
19     }
20 };
21 class Child : public Father, public Mother{ //Child inherits Father and Mother
22 public:
23     Child(int x):Mother(x), Father(x){
24         cout << "Child::Child(int) called" << endl;
25     }
26 };
27 int main(){
28     Child child(30);
29     return 0;
30 }
31
32 //Output
33 Person::Person() called
34 Father::Father(int) called
35 Mother::Mother(int) called
36 Child::Child(int) called
```

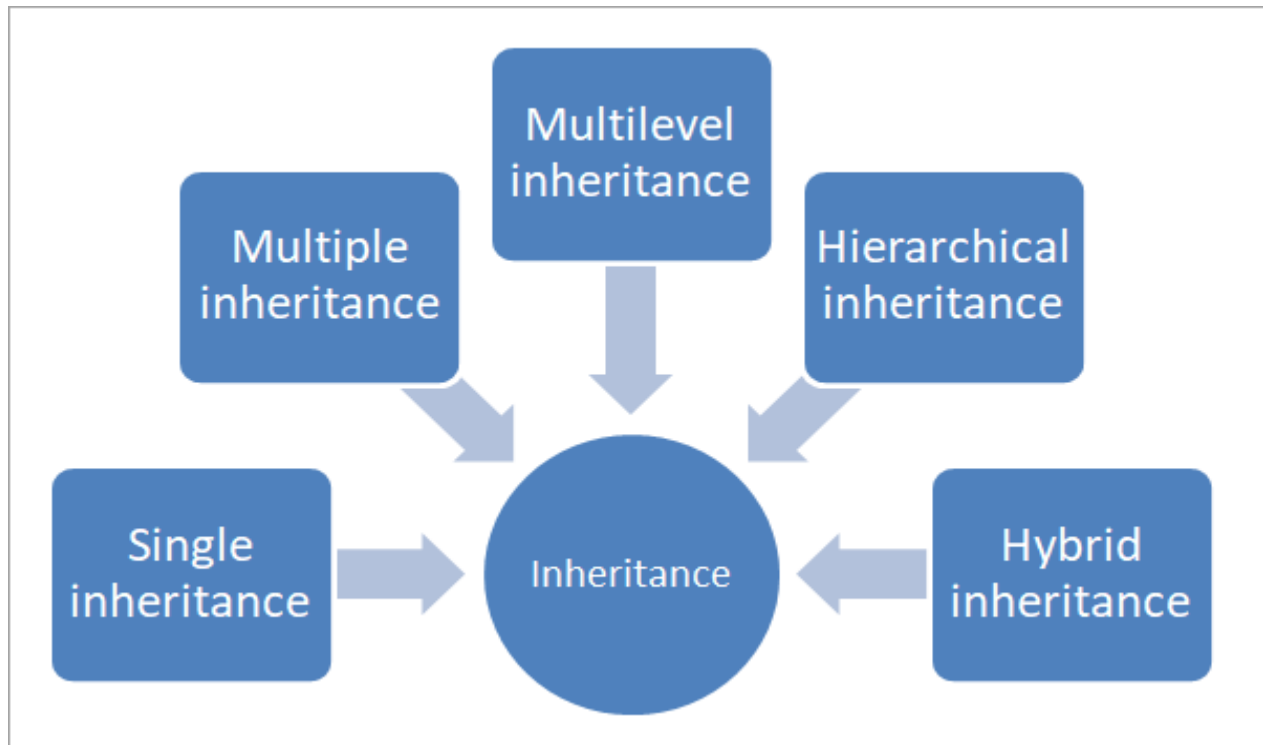
Uses-A, IsA and HasA Relationship :-



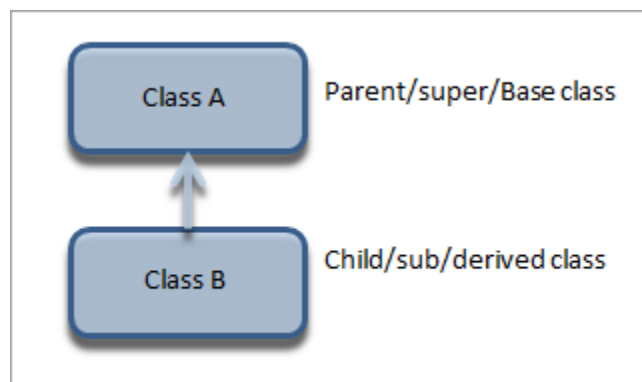
Access Specifiers in C++

Specifier	Within Same Class	In Derived Class	Outside the Class
Private	Yes	No	No
Protected	Yes	Yes	No
Public	Yes	Yes	Yes

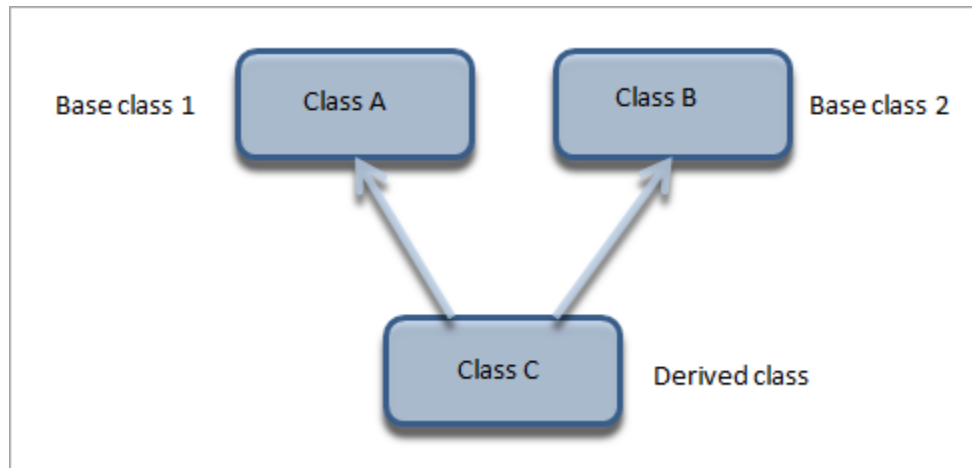
Types Of Inheritance :-



1) Single Inheritance :- In single inheritance, a class derives from one base class only. This means that there is only one subclass that is derived from one superclass.

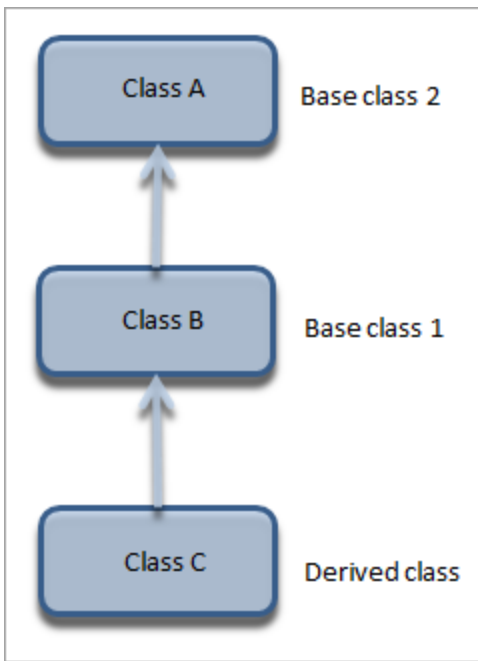


2) Multiple Inheritance :- Multiple inheritance is a type of inheritance in which a class derives from more than one class. As shown in the above diagram, class C is a subclass that has class A and class B as its parent.

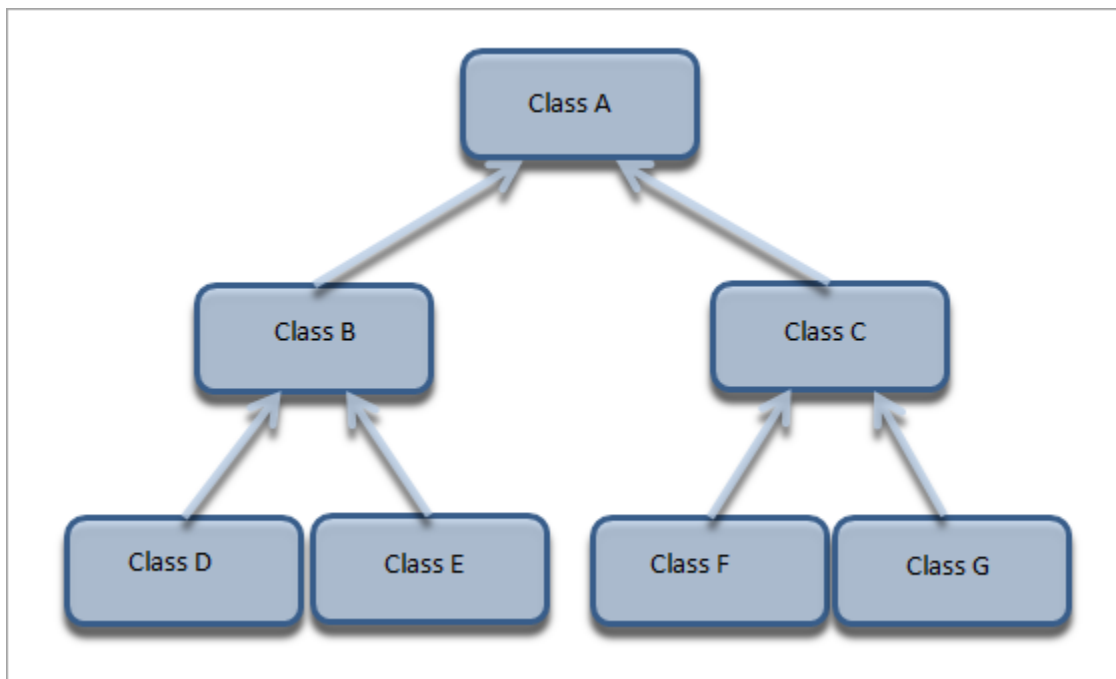


Example :- In a real-life scenario, a child inherits from their father and mother. This can be considered an example of multiple inheritance.

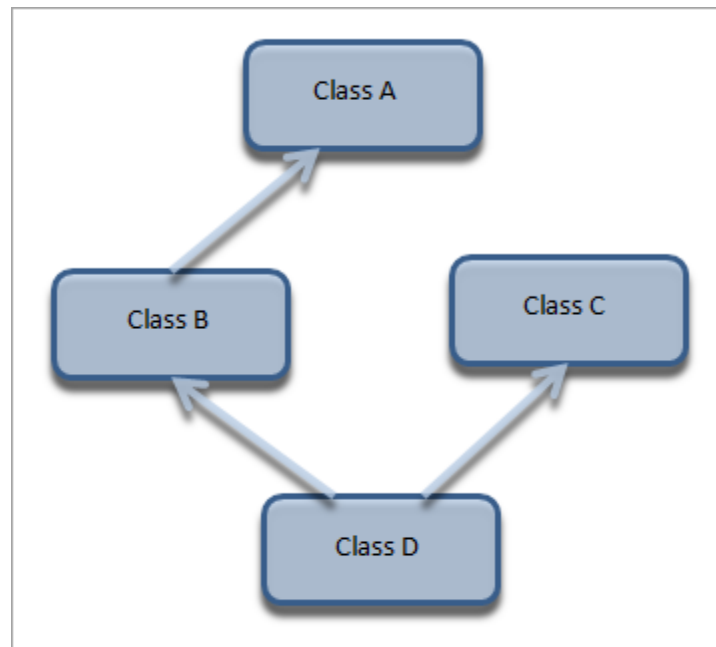
3) Multilevel Inheritance :- In multilevel inheritance, a class is derived from another derived class. This inheritance can have as many levels as long as our implementation doesn't go wayward. In the above diagram, class C is derived from Class B. Class B is in turn derived from class A.



4) Hierarchical Inheritance :- In hierarchical inheritance, more than one class inherits from a single base class as shown in the representation above. This gives it a structure of a hierarchy.



5) Hybrid Inheritance :- Hybrid inheritance is usually a combination of more than one type of inheritance. In the above representation, we have multiple inheritance (B, C, and D) and multilevel inheritance (A, B, and D) to get a hybrid inheritance.

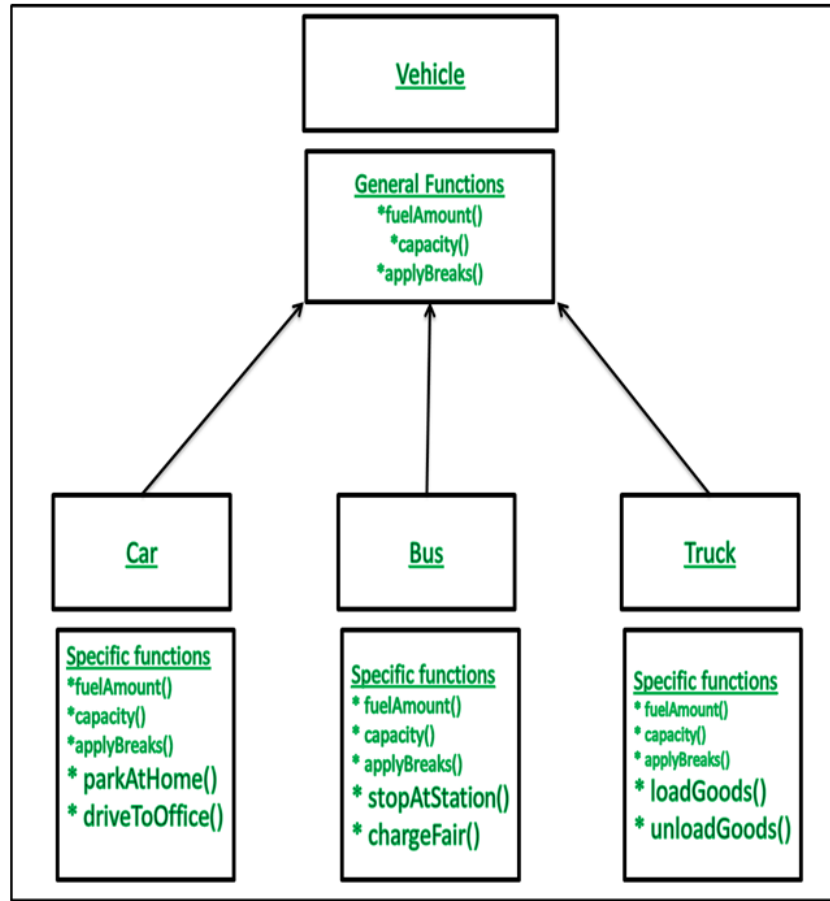


Access Specifier in Inheritance

```
1  #include<iostream>
2  using namespace std;
3  class Base {
4  public:
5      int x;
6  protected:
7      int y;
8  private:
9      int z;
10 };
11 class PublicDerived: public Base {
12     // x is public
13     // y is protected
14     // z is not accessible from PublicDerived
15 };
16 class ProtectedDerived: protected Base {
17     // x is protected
18     // y is protected
19     // z is not accessible from ProtectedDerived
20 };
21 class PrivateDerived: private Base {
22     // x is private
23     // y is private
24     // z is not accessible from PrivateDerived
25 };
```

Generalization

Up-Casting



Down-Casting

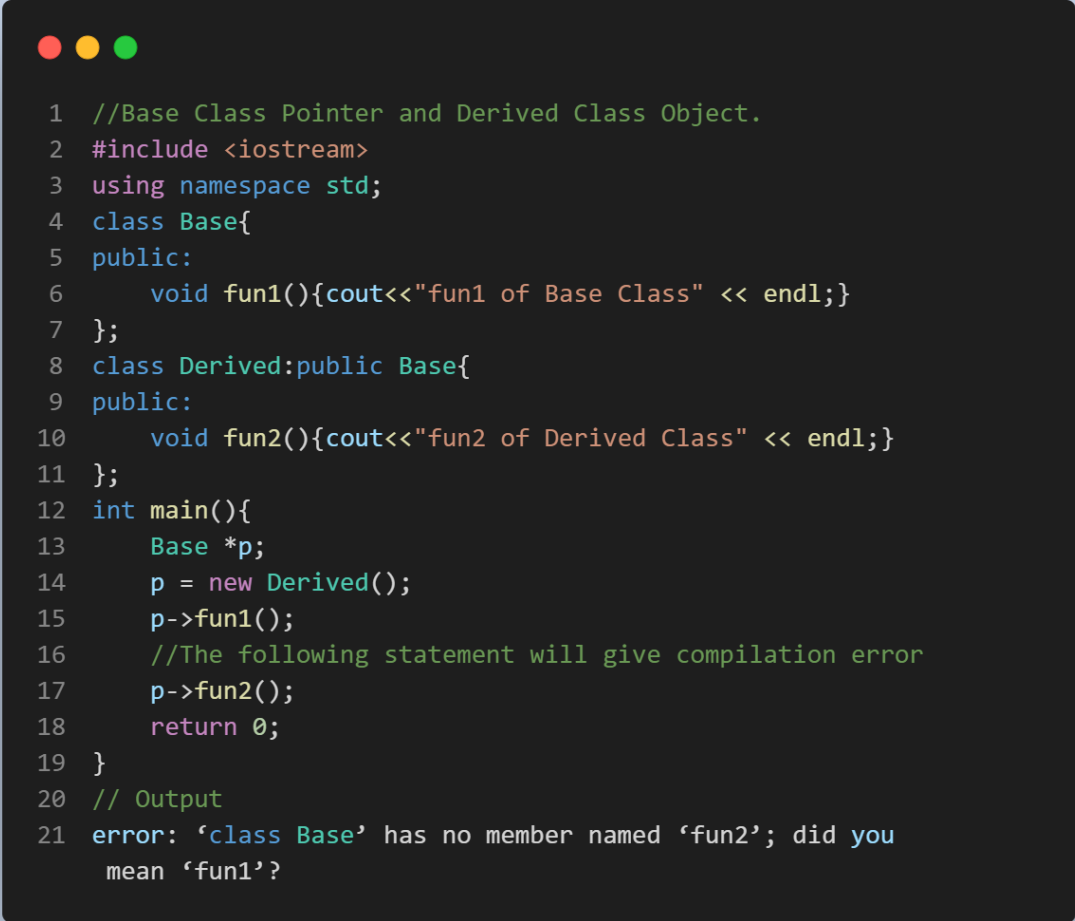
Specialization

Program to Demonstrate inheritance

```
1  // Program to Demonstrate Inheritance
2  #include <iostream>
3  using namespace std;
4  class Employee{
5  private:
6      int eid;
7      string name;
8  public:
9      Employee(int e, string n){
10         eid = e;
11         name = n;
12     }
13     int getEmployeeID(){return eid;}
14     string getName(){return name;}
15 };
16 class FulltimeEmployee : public Employee{
17 private:
18     int salary;
19 public:
20     FulltimeEmployee(int e, string n, int sal):Employee(e, n){
21         salary = sal;
22     }
23     int getSalary() { return salary; }
24 };
25 class ParttimeEmployee : public Employee{
26 private:
27     int wage;
28 public:
29     ParttimeEmployee(int e, string n, int w) : Employee(e, n){
30         wage = w;
31     }
32     int getWage() { return wage; }
33 };
34 int main(){
35     ParttimeEmployee p1(1, "John", 300);
36     FulltimeEmployee p2(2, "Raj", 5000);
37     cout<<"Salary of "<<p2.getName()<<" is "<<p2.getSalary()<<endl;
38     cout<<"Daily wage of "<<p1.getName()<<" is "<<p1.getWage()<<endl;
39 }
40
41 //Output
42 Salary of Raj is 5000
43 Daily wage of John is 300
```

Base Class Pointer and Derived Class Object in C++:-

```
Base * ptr;  
Ptr = new Derived();
```

A screenshot of a C++ code editor with a dark background and light-colored text. The code defines a base class 'Base' with a 'fun1()' method and a derived class 'Derived' that inherits from 'Base' and has its own 'fun2()' method. In the 'main()' function, a pointer of type 'Base' is created and points to a 'Derived' object. The code then calls 'fun1()' on the pointer, which works, but then attempts to call 'fun2()' on the same pointer, which results in a compilation error because the 'Base' class does not have a 'fun2()' method.

```
1 //Base Class Pointer and Derived Class Object.  
2 #include <iostream>  
3 using namespace std;  
4 class Base{  
5 public:  
6     void fun1(){cout<<"fun1 of Base Class" << endl;}  
7 };  
8 class Derived:public Base{  
9 public:  
10     void fun2(){cout<<"fun2 of Derived Class" << endl;}  
11 };  
12 int main(){  
13     Base *p;  
14     p = new Derived();  
15     p->fun1();  
16     //The following statement will give compilation error  
17     p->fun2();  
18     return 0;  
19 }  
20 // Output  
21 error: 'class Base' has no member named 'fun2'; did you  
    mean 'fun1'?
```

Eg. Suppose there is a basic Car. A basic car means having the basic features of a car. Nothing is automated, no extra features, except for driving a car there are no extra things like there is no air conditioner, no media player, and no keyless entry. Nothing is there.

Then you have an advanced car. And an advanced car is inheriting from a basic car. So, can we have a pointer of a basic car, and to that pointer, can we assign an object of an advanced car? Yes.

Then using the basic class pointer which functions you can call? You can call only the functions which are present in the basic car. It is just like a basic car but there is an advanced car here with all the extra features in the car which you have no idea about.

```
1 //Base Class Pointer and Derived Class Object.
2 #include<iostream>
3 using namespace std;
4 class BasicCar{
5 public:
6     void Start(){cout<<"Car Started"<<endl;}
7 };
8 class AdvanceCar:public BasicCar{
9 public:
10     void PlayMusic(){cout<<"Playing Music"<<endl;}
11 };
12
13 int main()
14 {
15     AdvanceCar a;
16     BasicCar *ptr = &a;
17     ptr->Start();
18     //The following statement will throw compilation error
19     ptr->PlayMusic();
20     return 0;
21 }
22
23 //Output
24 error: 'class BasicCar' has no member named 'PlayMusic'
```

Can we create a Derived pointer assigned to the Base class object in C++?

```
Derived *p;  
p = new Base();
```

Is it possible that we have a derived class pointer and I have assigned the object of the base class? **No, not possible.** Why? Let us see the reason.

See we have a basic car. **Now if we are calling the basic car as an advanced car.** Then can we get the advanced car features in the basic car? The answer is No.

So, many of the features of our advanced car are not available in the basic car. If I think that it is an advanced car, I cannot use the features. Suppose if I tried keyless entry then it is not there, if I try to open moon roof then it's not there, if I try to start AC then it's not there.